Agentic AI Mastery Roadmap — Task-Aligned Edition

> **Start: 10 Aug 2026 · End: 11 Jun 2027 · 2 hrs/day (Mon-Fri only)**
> **Target Role:** AI Engineer — LangGraph · CrewAI · RAG · FastAPI · Azure · Docker · LangSmith
> **Off Days:** Saturday, Sunday | **Hackathons:** 8 Aug 2026 (Sat) · 19-20 Oct 2026 (Mon-Tue)

Legend

Symbol	Meaning
📺	Watch / Read
💻	Code along
📝	Theory / Notes
🔨	Build / Implement (Worked Example / Practice / Challenge)
✅	Commit target for the day
🎯	Milestone Deliverable

Rule: One Task = One Day Minimum

- Har **Worked Example** apnay alag din pe hoga
- Har **Practice Task** apnay alag din pe hoga
- Har **Challenge** apnay alag din pe hoga
- **Deliverable** apnay alag din pe hoga
- Koi bhi din aik say zyada tasks nahi hoga

- Agar koi task zyada waqt ley to aglay din usay continue kero (buffer days hain har milestone ke baad)

Master Schedule

Milestone	Learning Days	Task Days	Total	Start	End
M0 — Advanced Python (PY4E Ch 11-16)	10	10	20	10 Aug 2026	04 Sep 2026
M1 — GenAI Foundations (Microsoft)	11	10	21	07 Sep 2026	05 Oct 2026
M2 — Agentic AI Basics + FastAPI	11	10	21	06 Oct 2026	05 Nov 2026
M3 — Microsoft AI Agents + LangGraph	14	10	24	06 Nov 2026	09 Dec 2026
M4 — CrewAI + HuggingFace Agents	12	10	22	10 Dec 2026	08 Jan 2027
M5 — RAG + Vector Databases	11	10	21	11 Jan 2027	08 Feb 2027
M6 — Advanced Multi-Agent + Azure	17	10	27	09 Feb 2027	17 Mar 2027
M7 — Orchestration + Docker + LangSmith	17	10	27	18 Mar 2027	23 Apr 2027
M8 — Capstone Projects	36	0	36	26 Apr 2027	11 Jun 2027

Total Working Days: 219
Total Calendar Span: 10 Aug 2026 → 11 Jun 2027 (306 days)

M0 — Advanced Python (PY4E Ch 11-16)

Duration: 20 days | **10 Aug → 04 Sep 2026**

> Tum already Chapter 11 pe ho. Ch 11-16 complete kero with agentic-AI-relevant focus.

📅 Weekly Objectives

Week	Dates	Days	Objective for the week
W1	10 Aug – 14 Aug	5	Ch 11 — Regex Basics; Ch 11 — Regex Advanced; Ch 12 — Networked Programs; Ch 12 — Parsing HTML; Ch 13 — XML & JSON
W2	17 Aug – 21 Aug	5	Ch 13 — API Security & Rate Limiting; Ch 14 — OOP; Ch 15 — Databases (SQLite); Ch 15 — SQL Advanced; Ch 16 — Data Visualization
W3	24 Aug – 28 Aug	5	Regex Log Parser; OOP Agent Simulator; SQLite Agent Memory; Email Extractor; JSON API Aggregator
W4	31 Aug – 04 Sep	5	Class Hierarchy for Tools; Recursive Web Crawler; Regex-based Config Parser; Thread-safe Agent Pool; Deliverable (milestone capstone)

Learning Phase (Days 1-10)

Day	Date	Topic	Activity	Commit Target
1	10 Aug	Ch 11 — Regex Basics	📺 Watch regex videos (re.search, re.findall, metacharacters) + 💻 Code along with `mbox-short.txt` examples	✅ `m0-d01-regex-basics.py` — extract emails from text file
2	11 Aug	Ch 11 — Regex Advanced	📝 Learn grouping, greedy vs non-greedy, `re.compile` + 💻 Parse log files	✅ `m0-d02-regex-advanced.py` — extract timestamps & IPs from logs
3	12 Aug	Ch 12 — Networked Programs	📺 HTTP protocol, sockets, `urllib` + 💻 Build a simple URL fetcher	✅ `m0-d03-url-fetcher.py` — fetch & display webpage headers
4	13 Aug	Ch 12 — Parsing HTML	📺 HTML parsing with BeautifulSoup + 💻 Extract all links from a page	✅ `m0-d04-link-extractor.py` — recursive link scraper (depth 1)
5	14 Aug	Ch 13 — XML & JSON	📺 XML parsing (`xml.etree`), JSON APIs + 💻 Consume a public API	✅ `m0-d05-api-client.py` — weather API fetcher with error handling
6	17 Aug	Ch 13 — API Security & Rate Limiting	📝 API keys, rate limits, headers + 💻 Build API client with retry logic	✅ `m0-d06-robust-api.py` — API client with exponential backoff

Day	Date	Topic	Activity	Commit Target
7	18 Aug	Ch 14 — OOP	📺 Classes, objects, `__init__`, inheritance, polymorphism + 💻 Design a `Agent` base class	✅ `m0-d07-oop-agent.py` — `BaseAgent` class with `run()` method
8	19 Aug	Ch 15 — Databases (SQLite)	📺 SQL basics, `sqlite3` module + 💻 CRUD operations	✅ `m0-d08-sqlite-crud.py` — agent memory store in SQLite
9	20 Aug	Ch 15 — SQL Advanced	📝 JOINs, indexes, transactions + 💻 Query optimization	✅ `m0-d09-sql-advanced.py` — multi-table agent log database
10	21 Aug	Ch 16 — Data Visualization	📺 `matplotlib` / `plotly` basics + 💻 Visualize agent performance metrics	✅ `m0-d10-data-viz.py` — bar chart of agent response times

Worked Examples Phase (Days 11-13) — 1 WE per day

Day	Date	Task	Activity	Commit Target
11	24 Aug	**WE 1: Regex Log Parser**	🔨 Parse Apache-style logs to extract IP, timestamp, status code, and user agent using `re.findall` with compiled patterns. Save structured output to CSV.	✅ `m0-we01-regex-log-parser/`
12	25 Aug	**WE 2: OOP Agent Simulator**	🔨 Build a `TaskAgent` class that inherits from `BaseAgent`, overrides `execute(task)` method, and maintains state. Create 3 agent instances and simulate task distribution.	✅ `m0-we02-oop-agent-sim/`
13	26 Aug	**WE 3: SQLite Agent Memory**	🔨 Create a database schema for agent conversations with tables: `agents`, `conversations`, `messages`. Implement insert, query, and delete operations with foreign key constraints.	✅ `m0-we03-sqlite-memory/`

Practice Tasks Phase (Days 14-16) — 1 PT per day

Day	Date	Task	Activity	Commit Target
14	27 Aug	**PT 1: Email Extractor**	🔨 Write a regex-based script that extracts all unique email addresses from a large text file and saves them sorted by domain. Handle edge cases like `+` aliases and subdomains.	✅ `m0-pt01-email-extractor/`
15	28 Aug	**PT 2: JSON API Aggregator**	🔨 Fetch data from 3 different public APIs, merge the results into a single JSON structure, and save to file with timestamp. Handle rate limits and failures gracefully.	✅ `m0-pt02-api-aggregator/`
16	31 Aug	**PT 3: Class Hierarchy for Tools**	🔨 Design an OOP hierarchy: `BaseTool` → `WebSearchTool`, `CalculatorTool`, `FileReaderTool`. Each implements `execute()` differently. Add a `ToolRegistry` that discovers and runs tools dynamically.	✅ `m0-pt03-tool-hierarchy/`

Challenges Phase (Days 17-19) — 1 CH per day

Day	Date	Task	Activity	Commit Target
17	01 Sep	**CH 1: Recursive Web Crawler**	🔨 Build a recursive web crawler (depth-limited to 2) that extracts all unique external links from a given domain. Handle timeouts, 404s, and robots.txt respectfully. Save results as a graph structure.	✅ `m0-ch01-web-crawler/`
18	02 Sep	**CH 2: Regex-based Config Parser**	🔨 Parse a custom `.ini`-style config file using only regex (no `configparser` module). Support sections, keys, values, comments, and multi-line values. Validate types (int, str, bool, list).	✅ `m0-ch02-config-parser/`
19	03 Sep	**CH 3: Thread-safe Agent Pool**	🔨 Create a pool of 5 `Agent` objects using threading. Distribute 20 tasks among them and collect results in a thread-safe SQLite database. Implement task queue with `queue.Queue` and handle deadlocks.	✅ `m0-ch03-agent-pool/`

Deliverable Phase (Day 20)

Day	Date	Task	Activity	Commit Target
20	04 Sep	**Deliverable**	🎯 **`agentic-python-toolkit`** — Polish and finalize GitHub repository containing all WE, PT, CH code plus unified README, architecture diagram, and setup instructions.	✅ `m0-deliverable/`

M1 — GenAI Foundations (Microsoft)

Duration: 21 days | **07 Sep → 05 Oct 2026**

> github.com/microsoft/generative-ai-for-beginners — 21 lessons

📅 Weekly Objectives

Week	Dates	Days	Objective for the week
W1	07 Sep – 11 Sep	5	L1-L2; L3-L4; L5-L6; L7-L8; L9-L10
W2	14 Sep – 18 Sep	5	L11-L12; L13-L14; L15-L16; L17-L18; L19-L20
W3	21 Sep – 25 Sep	5	L21 + Review; Prompt Engineering Suite; In-Memory RAG Pipeline; Semantic Kernel Plugin; Prompt Template Library
W4	28 Sep – 02 Oct	5	Embedding Similarity Search; Content Safety Filter; Multi-turn Conversation Memory; Dynamic Prompt Optimizer; RAG Evaluation Framework
W5	05 Oct	1	Deliverable (milestone capstone)

Learning Phase (Days 1-11)

Day	Date	Lessons	Activity	Commit Target
1	07 Sep	L1-L2	📺 Course intro, How GenAI works, LLMs basics	✅ `m1-d01-llm-basics.md` — LLM architecture summary
2	08 Sep	L3-L4	📺 Prompt engineering basics + 💻 Try prompts in OpenAI Playground	✅ `m1-d02-prompts.json` — 10 prompt examples with outputs
3	09 Sep	L5-L6	📺 Text generation, Tokenization + 📝 Build a simple text generator	✅ `m1-d03-text-gen.py` — basic prompt-to-text script
4	10 Sep	L7-L8	📺 Semantic Kernel intro, Building apps + 💻 Setup SK environment	✅ `m1-d04-sk-setup/` — Semantic Kernel hello-world
5	11 Sep	L9-L10	📺 Vector search, Embeddings + 📝 Theory notes on embeddings	✅ `m1-d05-embeddings.md` — embedding concepts & use-cases
6	14 Sep	L11-L12	📺 RAG fundamentals, Building RAG apps + 💻 Simple RAG pipeline	✅ `m1-d06-rag-basic.py` — in-memory RAG with text chunks
7	15 Sep	L13-L14	📺 Fine-tuning basics, RLHF + 📝 Notes on fine-tuning vs prompting	✅ `m1-d07-finetuning.md` — comparison table & when to use
8	16 Sep	L15-L16	📺 AI safety, Responsible AI + 📝 Build content safety checks	✅ `m1-d08-safety.py` — input/output filter script
9	17 Sep	L17-L18	📺 Multi-modal models, Image generation + 💻 Try DALL-E /	✅ `m1-d09-image-gen.py` — image generation script

Day	Date	Lessons	Activity	Commit Target
			Stable Diffusion API	
10	18 Sep	L19-L20	📺 LLM ops, Deployment basics + 📝 Architecture diagrams	✅ `m1-d10-llmops.md` — deployment checklist
11	21 Sep	L21 + Review	📺 Capstone lesson + 💻 Build mini-project + 📝 Review all 21 lessons	✅ `m1-d11-review/` — consolidated notes

Worked Examples Phase (Days 12-14)

Day	Date	Task	Activity	Commit Target
12	22 Sep	**WE 1: Prompt Engineering Suite**	🔨 Build a Python script with 5 prompt techniques: Zero-shot, Few-shot, Chain-of-Thought, Role-based, and Structured Output. Test each on the same 3 tasks and compare results in a markdown report.	✅ `m1-we01-prompt-suite/`
13	23 Sep	**WE 2: In-Memory RAG Pipeline**	🔨 Build a complete RAG system without external DB: chunk documents → embed with OpenAI/HuggingFace → cosine similarity retrieval → generate answer with context. Include source citation.	✅ `m1-we02-rag-pipeline/`
14	24 Sep	**WE 3: Semantic Kernel Plugin**	🔨 Create a custom SK plugin with 3 native functions: `SearchWeb`, `SummarizeText`, `ExtractEntities`. Chain them in a sequential planner and test with 5 real-world tasks.	✅ `m1-we03-sk-plugin/`

Practice Tasks Phase (Days 15-17)

Day	Date	Task	Activity	Commit Target
15	25 Sep	**PT 1: Prompt Template Library**	🔨 Create 10 reusable Jinja2/Mustache prompt templates for common agent tasks (summarization, extraction, classification, translation, code review). Include variable validation and default values.	✅ `m1-pt01-prompt-library/`
16	28 Sep	**PT 2: Embedding Similarity Search**	🔨 Generate embeddings for 50 sentences using `sentence-transformers`. Build a CLI tool that finds top-5 similar sentences for any query. Add cosine similarity threshold filtering.	✅ `m1-pt02-similarity-search/`
17	29 Sep	**PT 3: Content Safety Filter**	🔨 Implement a pre-filter and post-filter using keyword lists, regex patterns, and a simple sentiment threshold to block harmful inputs/outputs. Log all blocked attempts with reasons.	✅ `m1-pt03-safety-filter/`

Challenges Phase (Days 18-20)

Day	Date	Task	Activity	Commit Target
18	30 Sep	**CH 1: Multi-turn Conversation Memory**	🔨 Build a chatbot that maintains conversation history, summarizes past context when token limit approaches, and references previous answers. Test with 10-turn conversations.	✅ `m1-ch01-convo-memory/`
19	01 Oct	**CH 2: Dynamic Prompt Optimizer**	🔨 Create a system that takes a poorly performing prompt, generates 5 variations using meta-prompting, tests them all against a validation set, and returns the best-performing one with metrics.	✅ `m1-ch02-prompt-optimizer/`
20	02 Oct	**CH 3: RAG Evaluation Framework**	🔨 Build an evaluation script that measures RAG performance on 20 questions using metrics: context precision, answer relevance, and hallucination detection. Generate a visual report.	✅ `m1-ch03-rag-eval/`

Deliverable Phase (Day 21)

Day	Date	Task	Activity	Commit Target
21	05 Oct	**Deliverable**	🎯 **`genai-foundation-toolkit`** — Polish GitHub repo with all WE/PT/CH code, unified README with architecture diagram, setup guide, and demo screenshots.	✅ `m1-deliverable/`

M2 — Agentic AI Basics + FastAPI

Duration: 21 days | **06 Oct → 05 Nov 2026**

> Resources: Great Learning Agentic AI + FastAPI Tutorial

📅 Weekly Objectives

Week	Dates	Days	Objective for the week
W1	06 Oct – 12 Oct	5	Agentic AI — Planning & Reasoning; Agentic AI — Memory & Execution; Agentic AI — ReAct Pattern; Agentic AI — Tool Use; Agentic AI — Revie...
W2	13 Oct – 19 Oct	5	FastAPI — Path Ops & Basics; FastAPI — Pydantic Models; FastAPI — Async & Dependencies; FastAPI — Error Handling & Middleware; FastAPI — ...
W3	20 Oct – 26 Oct	5	FastAPI — Review & Integration; Agent API with FastAPI; Pydantic Validation Suite; Async Agent Orchestrator; ReAct Agent Implementation
W4	27 Oct – 02 Nov	5	FastAPI CRUD for Agent Memory; Dependency Injection Chain; Streaming Agent Responses; Rate-Limited Agent API; Multi-Agent Race Condition ...
W5	05 Nov	1	Deliverable (milestone capstone)

Learning Phase (Days 1-11)

Day	Date	Topic	Activity	Commit Target
1	06 Oct	Agentic AI — Planning & Reasoning	📺 Great Learning: Agent architecture, planning algorithms (ReAct, CoT, ToT)	✅ `m2-d01-agent-arch.md` — agent architecture notes
2	07 Oct	Agentic AI — Memory & Execution	📺 Memory types (short/long-term), tool use, execution loops	✅ `m2-d02-memory-patterns.md` — memory pattern comparison
3	08 Oct	Agentic AI — ReAct Pattern	📝 Study ReAct paper + 💻 Implement simple ReAct loop from scratch	✅ `m2-d03-react-loop.py` — ReAct pattern implementation
4	09 Oct	Agentic AI — Tool Use	💻 Build an agent that uses 2 tools: calculator + web search with decision logic	✅ `m2-d04-tool-agent.py` — tool-using agent
5	12 Oct	Agentic AI — Review + Mini Project	🔨 Build a simple goal-based agent with planning + 📝 Review all concepts	✅ `m2-d05-goal-agent.py` — goal-based agent with planning
6	13 Oct	FastAPI — Path Ops & Basics	📺 FastAPI tutorial: path params, query params, request body	✅ `m2-d06-fastapi-hello/` — basic FastAPI app
7	14 Oct	FastAPI — Pydantic Models	📺 Data validation, nested models, field types + 💻 Build schemas	✅ `m2-d07-pydantic-models.py` — agent request/response schemas
8	15 Oct	FastAPI — Async & Dependencies	📺 `async/await`, dependency injection,	✅ `m2-d08-async-deps.py` — async

Day	Date	Topic	Activity	Commit Target
			`Depends()` + 💻 Refactor app	endpoints with DI
9	16 Oct	FastAPI — Error Handling & Middleware	📺 Exception handlers, custom middleware, logging + 💻 Implement	✅ `m2-d09-middleware.py` — request logging middleware
10	19 Oct	FastAPI — Background Tasks & Testing	📺 Background tasks, startup/shutdown events + `TestClient`, pytest	✅ `m2-d10-background-tests.py` — background task + test suite
11	20 Oct	FastAPI — Review & Integration	📝 Review all FastAPI concepts + 💻 Integrate agent with FastAPI	✅ `m2-d11-fastapi-agent.py` — agent served via FastAPI

Worked Examples Phase (Days 12-14)

Day	Date	Task	Activity	Commit Target
12	21 Oct	**WE 1: Agent API with FastAPI**	🔨 Build a FastAPI app with endpoints: `POST /agent/run` (execute agent), `GET /agent/status/{id}` (check status), `POST /agent/tool` (register tool). Use Pydantic for all request/response models.	✅ `m2-we01-agent-api/`
13	22 Oct	**WE 2: Pydantic Validation Suite**	🔨 Create comprehensive Pydantic models for an agent system: `AgentConfig`, `ToolDefinition`, `RunRequest`, `RunResponse`, `MemoryEntry`. Include validators, custom errors, and nested model handling.	✅ `m2-we02-pydantic-suite/`
14	23 Oct	**WE 3: Async Agent Orchestrator**	🔨 Build an async FastAPI service that runs multiple agents concurrently using `asyncio.gather`, collects results, and returns aggregated responses with timing metrics per agent.	✅ `m2-we03-async-orch/`

Practice Tasks Phase (Days 15-17)

Day	Date	Task	Activity	Commit Target
15	26 Oct	**PT 1: ReAct Agent Implementation**	🔨 Implement a pure-Python ReAct agent with no framework. It should: receive a task → think → act (call tool) → observe → repeat until answer. Include max-iteration limit and cost tracking.	✅ `m2-pt01-pure-react/`
16	27 Oct	**PT 2: FastAPI CRUD for Agent Memory**	🔨 Build a complete CRUD API for agent memory storage using in-memory dict (later swappable with DB). Include pagination, filtering by agent_id, and sorting by timestamp.	✅ `m2-pt02-memory-crud/`
17	28 Oct	**PT 3: Dependency Injection Chain**	🔨 Create a dependency chain in FastAPI: `get_db` → `get_agent_service(db)` → `get_endpoint(service)`. Demonstrate proper DI patterns with 3 layers of abstraction.	✅ `m2-pt03-di-chain/`

Challenges Phase (Days 18-20)

Day	Date	Task	Activity	Commit Target
18	29 Oct	**CH 1: Streaming Agent Responses**	🔨 Implement a FastAPI endpoint that streams agent thought process in real-time using Server-Sent Events (SSE). The agent should stream each "Thought → Action → Observation" step to the client.	✅ `m2-ch01-sse-streaming/`
19	30 Oct	**CH 2: Rate-Limited Agent API**	🔨 Add custom rate limiting per API key using middleware. Different keys have different limits (free: 10/hr, pro: 100/hr). Store counters in Redis (or simulate with dict + TTL).	✅ `m2-ch02-rate-limiter/`
20	02 Nov	**CH 3: Multi-Agent Race Condition Handler**	🔨 Build an async system where 3 agents solve the same problem simultaneously. Return the fastest correct answer and cancel the others. Handle edge cases (all wrong, timeouts, partial answers).	✅ `m2-ch03-race-condition/`

Deliverable Phase (Day 21)

Day	Date	Task	Activity	Commit Target
21	05 Nov	**Deliverable**	🎯 **`agentic-fastapi-starter`** — Polish GitHub repo with complete FastAPI app, Pydantic schemas, ReAct agent, DI services, pytest suite with >80% coverage, and comprehensive README with curl examples.	✅ `m2-deliverable/`

M3 — Microsoft AI Agents + LangGraph

Duration: 24 days | **06 Nov → 09 Dec 2026**

> Resources: Microsoft AI Agents for Beginners (12 lessons) + DeepLearning.AI LangGraph

📅 Weekly Objectives

Week	Dates	Days	Objective for the week
W1	06 Nov – 12 Nov	5	MS L1-L2; MS L3-L4; MS L5-L6; MS L7-L8; MS L9-L10
W2	13 Nov – 19 Nov	5	MS L11-L12; LangGraph — State & Graphs; LangGraph — Tools; LangGraph — Memory; LangGraph — Human-in-the-Loop
W3	20 Nov – 26 Nov	5	LangGraph — Streaming & Subgraphs; LangGraph — Multi-Agent; LangGraph — Evaluation; LangGraph — Review & Integration; Semantic Kernel Res...
W4	27 Nov – 03 Dec	5	LangGraph State Machine for Customer Support; Multi-Agent Supervisor with LangGraph; AutoGen Coding Duo; LangGraph RAG with Web Search Fa...
W5	04 Dec – 09 Dec	4	Dynamic Tool Discovery & Registration; Self-Reflecting Agent Loop; Cross-Framework Agent Bridge; Deliverable (milestone capstone)

Learning Phase (Days 1-14)

Day	Date	Topic	Activity	Commit Target
1	06 Nov	MS L1-L2	📺 Agent basics, Semantic Kernel intro, agent types	✅ `m3-d01-sk-basics.md` — SK architecture notes
2	06 Nov	MS L3-L4	📺 AutoGen intro, multi-agent patterns, conversable agents	✅ `m3-d02-autogen.md` — AutoGen pattern notes
3	10 Nov	MS L5-L6	📺 Azure AI Foundry, agent deployment, model catalog	✅ `m3-d03-azure-foundry.md` — Azure AI notes
4	11 Nov	MS L7-L8	📺 Tool use, function calling + 💻 Code along with function definitions	✅ `m3-d04-function-calling.py` — function calling examples
5	12 Nov	MS L9-L10	📺 RAG agents, memory management + 💻 Build RAG agent with SK	✅ `m3-d05-rag-agent.py` — SK RAG implementation
6	13 Nov	MS L11-L12	📺 Evaluation, monitoring, responsible AI + 📝 Review all 12 lessons	✅ `m3-d06-ms-review.md` — complete MS course summary
7	16 Nov	LangGraph — State & Graphs	📺 State management, graph structure, nodes & edges	✅ `m3-d07-lg-state.py` — basic graph with 3 nodes
8	17 Nov	LangGraph — Tools	📺 Tool binding, tool nodes, `ToolNode` + 💻 Build tool node	✅ `m3-d08-lg-tools.py` — tool integration in graph

Day	Date	Topic	Activity	Commit Target
9	18 Nov	LangGraph — Memory	📺 Short-term & long-term memory, checkpointing with SQLite	✅ `m3-d09-lg-memory.py` — persistent memory in graph
10	19 Nov	LangGraph — Human-in-the-Loop	📺 Interrupts, approvals, editing state before continue	✅ `m3-d10-lg-human.py` — HITL approval pattern
11	20 Nov	LangGraph — Streaming & Subgraphs	📺 Stream tokens, stream steps, subgraph composition	✅ `m3-d11-lg-stream-subgraph.py` — streaming + modular graphs
12	23 Nov	LangGraph — Multi-Agent	📺 Multi-agent patterns: supervisor, swarm, hierarchical	✅ `m3-d12-lg-multi.py` — supervisor pattern implementation
13	24 Nov	LangGraph — Evaluation	📺 LangSmith evals, dataset creation, batch testing	✅ `m3-d13-lg-eval.py` — evaluation setup with datasets
14	25 Nov	LangGraph — Review & Integration	📝 Review all LangGraph concepts + 💻 Build integrated example	✅ `m3-d14-lg-review.py` — comprehensive review project

Worked Examples Phase (Days 15-17)

Day	Date	Task	Activity	Commit Target
15	26 Nov	**WE 1: Semantic Kernel Research Agent**	🔨 Build an SK agent that: receives a topic → searches web → summarizes findings → stores in memory → answers follow-up questions. Use SK plugins, planners, and memory connectors.	✅ `m3-we01-sk-research/`
16	27 Nov	**WE 2: LangGraph State Machine for Customer Support**	🔨 Build a LangGraph with states: `intake` → `diagnose` → `resolve` → `escalate` → `close`. Each state has conditional edges based on sentiment analysis and confidence scores.	✅ `m3-we02-lg-support/`
17	30 Nov	**WE 3: Multi-Agent Supervisor with LangGraph**	🔨 Implement a supervisor pattern: 3 worker agents (researcher, writer, critic) + 1 supervisor. Supervisor assigns tasks, reviews output, and decides when to finalize. Include max iterations.	✅ `m3-we03-lg-supervisor/`

Practice Tasks Phase (Days 18-20)

Day	Date	Task	Activity	Commit Target
18	01 Dec	**PT 1: AutoGen Coding Duo**	🔨 Create two AutoGen agents: a "Coder" and a "Reviewer". They iteratively improve a Python function until the Reviewer approves. Log all conversation turns and final code diff.	✅ `m3-pt01-autogen-duo/`
19	02 Dec	**PT 2: LangGraph RAG with Web Search Fallback**	🔨 Build a LangGraph agent that first tries to answer from a local vector store; if confidence is low, it falls back to web search, then updates the vector store with new findings.	✅ `m3-pt02-lg-rag-fallback/`
20	03 Dec	**PT 3: Human-in-the-Loop Expense Approval**	🔨 Build an expense reporting agent in LangGraph. It extracts expense details from receipt text, calculates totals, and pauses for human approval before submitting. Allow editing of extracted data.	✅ `m3-pt03-lg-expense/`

Challenges Phase (Days 21-23)

Day	Date	Task	Activity	Commit Target
21	04 Dec	**CH 1: Dynamic Tool Discovery & Registration**	🔨 Build a system where agents can discover available tools at runtime from a registry (JSON/YAML), register them dynamically, and use them without hardcoded definitions. Include tool schema validation.	✅ `m3-ch01-dynamic-tools/`
22	07 Dec	**CH 2: Self-Reflecting Agent Loop**	🔨 Create a LangGraph agent that solves a problem, then reflects on its own answer using a separate "critic" node. If the critic finds flaws, the graph loops back to revision (max 3 iterations). Track improvement metrics.	✅ `m3-ch02-self-reflect/`
23	08 Dec	**CH 3: Cross-Framework Agent Bridge**	🔨 Build a bridge that allows a Semantic Kernel agent and a LangGraph agent to communicate. SK agent delegates subtasks to LangGraph agent and receives structured results back via a shared message bus.	✅ `m3-ch03-framework-bridge/`

Deliverable Phase (Day 24)

Day	Date	Task	Activity	Commit Target
24	09 Dec	**Deliverable**	🎯 `langgraph-agent-suite` — Polish GitHub repo with SK research agent, 3 LangGraph implementations (state machine, multi-agent, RAG), AutoGen coding duo, and comprehensive README with architecture diagrams.	✅ `m3-deliverable/`

M4 — CrewAI + HuggingFace Agents

Duration: 22 days | **10 Dec → 08 Jan 2027**

> Resources: CrewAI DeepLearning.AI + HF Agents Course

Weekly Objectives

Week	Dates	Days	Objective for the week
W1	10 Dec – 16 Dec	5	CrewAI — Basics; CrewAI — Tools; CrewAI — Processes; CrewAI — Memory & Guardrails; CrewAI — Multi-Crew Pipelines
W2	17 Dec – 23 Dec	5	CrewAI — Review & Project; HF — Unit 0-1; HF — Unit 2; HF — Unit 3; HF — Unit 4
W3	24 Dec – 30 Dec	5	HF — Unit 5-6; HF — Certification Prep; CrewAI Content Creation Team; smolagents Code Execution Agent; HuggingFace Spaces Deployment
W4	31 Dec – 06 Jan	5	CrewAI Market Research Crew; smolagents Financial Calculator; LlamaIndex Document Q&A Agent; CrewAI + LangGraph Bridge; Self-Improving sm...
W5	07 Jan – 08 Jan	2	Multi-Framework Orchestrator; Deliverable (milestone capstone)

Learning Phase (Days 1-12)

Day	Date	Topic	Activity	Commit Target
1	10 Dec	CrewAI — Basics	📺 CrewAI intro, agents, tasks, crews configuration	✅ `m4-d01-crewai-basics.py` — first crew with 2 agents
2	11 Dec	CrewAI — Tools	📺 Custom tools, tool delegation, `@tool` decorator + 💻 Build 2 tools	✅ `m4-d02-crewai-tools.py` — custom tool implementation
3	14 Dec	CrewAI — Processes	📺 Sequential vs hierarchical processes + 💻 Implement both patterns	✅ `m4-d03-crewai-processes.py` — process comparison
4	15 Dec	CrewAI — Memory & Guardrails	📺 Short-term, long-term, entity memory + guardrails & callbacks	✅ `m4-d04-crewai-memory.py` — memory-enabled crew
5	16 Dec	CrewAI — Multi-Crew Pipelines	📺 Crew-to-crew delegation, pipelines, output routing	✅ `m4-d05-crewai-pipelines.py` — multi-crew system
6	17 Dec	CrewAI — Review & Project	📝 Review all CrewAI concepts + 💻 Build mini crew project	✅ `m4-d06-crewai-review.py` — comprehensive review
7	18 Dec	HF — Unit 0-1	📺 smolagents intro, agent anatomy, code agents, `CodeAgent`	✅ `m4-d07-hf-intro.md` — smolagents fundamentals
8	21 Dec	HF — Unit 2	📺 Tool creation, tool calling, managed agents, `Tool` class	✅ `m4-d08-hf-tools.py` — custom tool for

Day	Date	Topic	Activity	Commit Target
				smolagents
9	22 Dec	HF — Unit 3	📺 LlamaIndex agents, RAG agents, query engines + 💻 Build RAG agent	✅ `m4-d09-hf-llamaindex.py` — LlamaIndex agent
10	23 Dec	HF — Unit 4	📺 LangGraph integration with HF ecosystem, shared patterns	✅ `m4-d10-hf-langgraph.py` — HF + LangGraph bridge
11	24 Dec	HF — Unit 5-6	📺 Evaluation, observability, testing + Deployment to HF Spaces, Gradio UI	✅ `m4-d11-hf-deploy.py` — deployed HF Space
12	25 Dec	HF — Certification Prep	📝 Review all units, practice quizzes, fill knowledge gaps	✅ `m4-d12-cert-prep.md` — certification readiness

Worked Examples Phase (Days 13-15)

Day	Date	Task	Activity	Commit Target
13	28 Dec	**WE 1: CrewAI Content Creation Team**	🔨 Build a crew with 4 agents: `Researcher` (gathers info), `OutlineWriter` (creates structure), `ContentWriter` (writes article), `Editor` (reviews & refines). Use sequential process with memory and guardrails.	✅ `m4-we01-crewai-content/`
14	29 Dec	**WE 2: smolagents Code Execution Agent**	🔨 Build a `smolagents` agent that receives a natural language request, writes Python code to solve it, executes the code safely in a sandbox (restricted `exec`), and returns the result. Include error handling and retry.	✅ `m4-we02-smolagent-code/`
15	30 Dec	**WE 3: HuggingFace Spaces Deployment**	🔨 Deploy a multi-tool agent to HF Spaces with Gradio UI. The agent should have: text input, file upload, web search, and code execution capabilities. Write proper `README.md` and `app.py`.	✅ `m4-we03-hf-deployed/`

Practice Tasks Phase (Days 16-18)

Day	Date	Task	Activity	Commit Target
16	31 Dec	**PT 1: CrewAI Market Research Crew**	🔨 Build a hierarchical crew for market research: Manager agent delegates to `DataCollector`, `Analyst`, and `Reporter`. Use `process="hierarchical"` with `manager_llm`. Generate a PDF report.	✅ `m4-pt01-research-crew/`
17	01 Jan	**PT 2: smolagents Financial Calculator**	🔨 Create a smolagent with custom tools: `get_stock_price`, `calculate_rsi`, `plot_chart`. The agent answers financial queries using these tools with reasoning steps and citations.	✅ `m4-pt02-finance-agent/`
18	04 Jan	**PT 3: LlamaIndex Document Q&A Agent**	🔨 Build a LlamaIndex agent that ingests PDFs, creates a vector index, and answers questions with source citations. Include re-ranking with `bge-reranker` for better retrieval.	✅ `m4-pt03-llama-rag/`

Challenges Phase (Days 19-21)

Day	Date	Task	Activity	Commit Target
19	05 Jan	**CH 1: CrewAI + LangGraph Bridge**	🔨 Create a system where a CrewAI crew delegates a specialized subtask to a LangGraph subgraph, receives the result, and continues the crew process seamlessly. Use a shared state object.	✅ `m4-ch01-crewai-lg-bridge/`
20	06 Jan	**CH 2: Self-Improving smolagent**	🔨 Build a smolagent that evaluates its own tool usage performance, maintains a "lessons learned" memory (JSON file), and adjusts its tool selection strategy over multiple runs based on success rates.	✅ `m4-ch02-self-improving/`
21	07 Jan	**CH 3: Multi-Framework Orchestrator**	🔨 Design an orchestrator that can dispatch tasks to the right framework (CrewAI for role-based tasks, LangGraph for state-machine tasks, smolagents for code tasks) based on task classification using an LLM router.	✅ `m4-ch03-orchestrator/`

Deliverable Phase (Day 22)

Day	Date	Task	Activity	Commit Target
22	08 Jan	**Deliverable**	🎯 **`multi-framework-agent-suite`** — Polish GitHub repo + HF Space with CrewAI crews, smolagents, LlamaIndex RAG, multi-framework orchestrator, deployed Gradio app, and comprehensive README with demo videos.	✅ `m4-deliverable/`

M5 — RAG + Vector Databases

Duration: 21 days | **11 Jan → 08 Feb 2027**

> Resources: LangChain RAG from Scratch (16 videos) + Pinecone Quickstart

📅 Weekly Objectives

Week	Dates	Days	Objective for the week
W1	11 Jan – 15 Jan	5	RAG — Chunking Strategies; RAG — Embeddings Deep Dive; RAG — Vector DB Intro; RAG — Retrieval Basics; RAG — Advanced Retrieval
W2	18 Jan – 22 Jan	5	RAG — Generation & Prompting; RAG — Evaluation; RAG — End-to-End Pipeline; Pinecone — Setup & Basics; Pinecone — Metadata & Filtering
W3	25 Jan – 29 Jan	5	Pinecone — Integration & Review; Advanced Chunking Pipeline; Hybrid RAG with Re-ranking; RAG Evaluation Framework; Multi-modal RAG
W4	01 Feb – 05 Feb	5	Conversational RAG with Memory; Agentic RAG; Self-Healing RAG; Real-time RAG with Streaming Updates; Multi-tenant RAG API
W5	08 Feb	1	Deliverable (milestone capstone)

Learning Phase (Days 1-11)

Day	Date	Topic	Activity	Commit Target
1	11 Jan	RAG — Chunking Strategies	📺 Video 1-2: Text splitting, recursive chunking, semantic chunking, character vs token-based	✅ `m5-d01-chunking.py` — compare 3 chunkers on same doc
2	12 Jan	RAG — Embeddings Deep Dive	📺 Video 3-4: Embedding models (OpenAI, HF, Cohere), comparison, dimensionality, fine-tuning	✅ `m5-d02-embeddings.py` — embedding model comparison
3	13 Jan	RAG — Vector DB Intro	📺 Video 5-6: Vector DB concepts, similarity search, ANN algorithms (HNSW, IVF)	✅ `m5-d03-vectordb.md` — vector DB comparison table
4	14 Jan	RAG — Retrieval Basics	📺 Video 7-8: Similarity search, MMR, metadata filtering, top-k vs threshold	✅ `m5-d04-retrieval.py` — basic retrieval with filtering
5	15 Jan	RAG — Advanced Retrieval	📺 Video 9-10: Hybrid search, re-ranking, query expansion, hypothetical document embedding	✅ `m5-d05-advanced-retrieval.py` — hybrid search implementation
6	18 Jan	RAG — Generation & Prompting	📺 Video 11-12: Context stuffing, citation, answer synthesis, handling no-context	✅ `m5-d06-generation.py` — citation-aware generation

Day	Date	Topic	Activity	Commit Target
7	19 Jan	RAG — Evaluation	📺 Video 13-14: RAGAS metrics, context precision, faithfulness, answer relevance	✅ `m5-d07-evaluation.py` — RAG metrics calculation
8	20 Jan	RAG — End-to-End Pipeline	📺 Video 15-16: Full pipeline, optimization, caching strategies, cost reduction	✅ `m5-d08-pipeline.py` — optimized end-to-end RAG
9	21 Jan	Pinecone — Setup & Basics	📺 Quickstart: index creation, upsert, query, namespaces, dimension settings	✅ `m5-d09-pinecone-basics.py` — Pinecone hello-world
10	22 Jan	Pinecone — Metadata & Filtering	📺 Metadata filters, namespaces, hybrid search, sparse-dense vectors	✅ `m5-d10-pinecone-advanced.py` — metadata filtering
11	25 Jan	Pinecone — Integration & Review	💻 Integrate Pinecone with LangChain RAG pipeline + 📝 Review all RAG concepts	✅ `m5-d11-pinecone-rag.py` — full Pinecone + LangChain integration

Worked Examples Phase (Days 12-14)

Day	Date	Task	Activity	Commit Target
12	26 Jan	**WE 1: Advanced Chunking Pipeline**	🔨 Implement 5 chunking strategies (fixed-size, recursive, semantic, agentic, hierarchical) on the same document. Evaluate each with retrieval accuracy metrics and visualize chunk boundaries with matplotlib.	✅ `m5-we01-chunking-pipeline/`
13	27 Jan	**WE 2: Hybrid RAG with Re-ranking**	🔨 Build a RAG system that combines BM25 keyword search + dense vector search. Use a cross-encoder re-ranker (like `bge-reranker`) on the top-20 candidates to return top-5. Compare against baseline with metrics.	✅ `m5-we02-hybrid-rag/`
14	28 Jan	**WE 3: RAG Evaluation Framework**	🔨 Create a comprehensive evaluation suite using RAGAS metrics: faithfulness, answer relevancy, context precision, context recall. Run on 50 test questions and generate a report with charts and recommendations.	✅ `m5-we03-rag-eval/`

Practice Tasks Phase (Days 15-17)

Day	Date	Task	Activity	Commit Target
15	29 Jan	**PT 1: Multi-modal RAG**	🔨 Build a RAG system that handles both text and images. Use CLIP embeddings for images, text embeddings for documents. Answer questions that require cross-referencing images and text with source attribution.	✅ `m5-pt01-multimodal-rag/`
16	01 Feb	**PT 2: Conversational RAG with Memory**	🔨 Implement a RAG chatbot that maintains conversation history. It should rephrase follow-up questions using context, retrieve relevant docs, and generate contextual answers with source links and confidence scores.	✅ `m5-pt02-convo-rag/`
17	02 Feb	**PT 3: Agentic RAG**	🔨 Build an agent that decides whether to: (a) answer from memory, (b) search vector DB, (c) search web, or (d) ask clarifying question. Implement the decision logic with confidence thresholds and tool use.	✅ `m5-pt03-agentic-rag/`

Challenges Phase (Days 18-20)

Day	Date	Task	Activity	Commit Target
18	03 Feb	**CH 1: Self-Healing RAG**	🔨 Create a RAG system that detects when retrieval quality is poor (low confidence, no relevant chunks) and automatically switches strategies: query expansion → synonym expansion → web fallback. Log all strategy switches.	✅ `m5-ch01-self-healing/`
19	04 Feb	**CH 2: Real-time RAG with Streaming Updates**	🔨 Build a RAG pipeline that monitors a data source (e.g., RSS feed, API) and automatically updates the vector index when new documents arrive. Ensure zero-downtime updates with versioning.	✅ `m5-ch02-realtime-rag/`
20	05 Feb	**CH 3: Multi-tenant RAG API**	🔨 Design a RAG API where each user has isolated namespaces in Pinecone, separate embedding caches, and role-based access. Implement with FastAPI + Pinecone metadata filtering + API key authentication.	✅ `m5-ch03-multitenant-rag/`

Deliverable Phase (Day 21)

Day	Date	Task	Activity	Commit Target
21	08 Feb	**Deliverable**	🎯 **`production-rag-system`** — Polish GitHub repo with chunking comparison, hybrid search + re-ranking, RAGAS evaluation suite, Pinecone integration with metadata filtering, FastAPI RAG endpoints, and benchmark report.	✅ `m5-deliverable/`

M6 — Advanced Multi-Agent + Azure

Duration: 27 days | **09 Feb → 17 Mar 2027**

> Resources: IBM Coursera + Microsoft Learn Azure

📅 Weekly Objectives

Week	Dates	Days	Objective for the week
W1	09 Feb – 15 Feb	5	IBM — Course Intro & BeeAI; IBM — LangGraph Advanced Patterns; IBM — CrewAI Advanced; IBM — AutoGen/AG2; IBM — Multi-Agent Patterns
W2	16 Feb – 22 Feb	5	IBM — Project Work (Part 1); IBM — Project Work (Part 2); IBM — Project Work (Part 3); IBM — Review & Catch-up; Azure — OpenAI Service
W3	23 Feb – 01 Mar	5	Azure — Prompt Flow; Azure — Content Safety; Azure — AI Foundry; Azure — SK on Azure; Azure — Agent Deployment
W4	02 Mar – 08 Mar	5	Azure — Monitoring & Insights; Azure — Hands-on Lab & Review; Parallel Voter Pattern; Azure Prompt Flow Agent; Azure SK + LangGraph Hybrid
W5	09 Mar – 15 Mar	5	AG2 Debate System; Azure RAG Agent Deployment; Multi-Cloud Agent; Fault-Tolerant Agent Swarm; Azure CI/CD Pipeline
W6	16 Mar – 17 Mar	2	Cross-Region Agent with Latency Optimization; Deliverable (milestone capstone)

Learning Phase (Days 1-17)

Day	Date	Topic	Activity	Commit Target
1	09 Feb	IBM — Course Intro & BeeAI	📺 Agentic AI landscape, BeeAI framework, course overview	✅ `m6-d01-ibm-intro.md` — landscape analysis
2	10 Feb	IBM — LangGraph Advanced Patterns	📺 Sequential, parallel, routing, map-reduce patterns in LangGraph	✅ `m6-d02-ibm-langgraph.py` — advanced pattern implementations
3	11 Feb	IBM — CrewAI Advanced	📺 Advanced crew patterns, custom managers, callback systems	✅ `m6-d03-ibm-crewai.py` — advanced crew configurations
4	12 Feb	IBM — AutoGen/AG2	📺 AG2 framework, conversable agents, group chat, nested chats	✅ `m6-d04-ibm-autogen.py` — AG2 group chat example
5	15 Feb	IBM — Multi-Agent Patterns	📺 Pattern library: router, aggregator, voter, debater, pipeline	✅ `m6-d05-ibm-patterns.py` — pattern implementations
6	16 Feb	IBM — Project Work (Part 1)	💻 Start IBM course project: design multi-agent system architecture	✅ `m6-d06-ibm-project-design.md` — architecture document
7	17 Feb	IBM — Project Work (Part 2)	💻 Implement core agents and orchestration logic	✅ `m6-d07-ibm-project-core/` — core implementation

Day	Date	Topic	Activity	Commit Target
8	18 Feb	IBM — Project Work (Part 3)	💻 Add tools, memory, evaluation, and polish	✅ `m6-d08-ibm-project-final/` — complete project
9	19 Feb	IBM — Review & Catch-up	📝 Review all IBM modules, fill gaps, refactor code	✅ `m6-d09-ibm-review.md` — comprehensive review
10	22 Feb	Azure — OpenAI Service	📺 Azure OpenAI setup, model deployments, endpoints, keys	✅ `m6-d10-azure-openai.md` — Azure OpenAI setup guide
11	23 Feb	Azure — Prompt Flow	📺 Prompt flow designer, connections, variants, evaluation flows	✅ `m6-d11-azure-promptflow.py` — prompt flow implementation
12	24 Feb	Azure — Content Safety	📺 Content safety filters, responsible AI, harm categories	✅ `m6-d12-azure-safety.py` — safety filter implementation
13	25 Feb	Azure — AI Foundry	📺 Foundry hub, projects, model catalog, fine-tuning jobs	✅ `m6-d13-azure-foundry.md` — Foundry project setup
14	26 Feb	Azure — SK on Azure	📺 Semantic Kernel + Azure OpenAI integration, kernel setup	✅ `m6-d14-azure-sk.py` — SK + Azure integration
15	01 Mar	Azure — Agent Deployment	📺 Deploy agents to Azure Container Apps, scaling, networking	✅ `m6-d15-azure-deploy.md` — deployment configuration

Day	Date	Topic	Activity	Commit Target
16	02 Mar	Azure — Monitoring & Insights	📺 Application Insights, tracing, logging, custom metrics	✅ `m6-d16-azure-monitor.py` — monitoring setup
17	03 Mar	Azure — Hands-on Lab & Review	💻 Full Azure agent deployment lab + 📝 Review all Azure concepts	✅ `m6-d17-azure-lab/` — complete deployment lab

Worked Examples Phase (Days 18-20)

Day	Date	Task	Activity	Commit Target
18	04 Mar	**WE 1: Parallel Voter Pattern**	🔨 Implement a parallel voter pattern in LangGraph: 5 agents solve the same problem independently → aggregator compiles votes → majority answer returned with confidence score. Handle ties with a tie-breaker agent.	✅ `m6-we01-parallel-voter/`
19	05 Mar	**WE 2: Azure Prompt Flow Agent**	🔨 Build a complete agent in Azure AI Foundry's Prompt Flow. Include: prompt variants, evaluation flow, content safety node, and deployment to an endpoint. Test with real queries and log results.	✅ `m6-we02-azure-pf/`
20	08 Mar	**WE 3: Azure SK + LangGraph Hybrid**	🔨 Create a hybrid system: LangGraph orchestrates the workflow, but LLM calls route through Azure OpenAI via Semantic Kernel. Implement token counting and cost tracking per request.	✅ `m6-we03-azure-hybrid/`

Practice Tasks Phase (Days 21-23)

Day	Date	Task	Activity	Commit Target
21	09 Mar	**PT 1: AG2 Debate System**	🔨 Build a 3-agent debate in AG2: `Proponent`, `Opponent`, `Moderator`. They debate a given topic for 3 rounds. Moderator summarizes arguments and declares a winner based on evidence quality and logical consistency.	✅ `m6-pt01-ag2-debate/`
22	10 Mar	**PT 2: Azure RAG Agent Deployment**	🔨 Build a RAG agent locally, containerize it with Docker, deploy to Azure Container Apps with Azure OpenAI backend, and configure auto-scaling rules based on request queue depth.	✅ `m6-pt02-azure-rag/`
23	11 Mar	**PT 3: Multi-Cloud Agent**	🔨 Design an agent that can switch between Azure OpenAI and OpenAI based on availability, cost, or task type. Implement a router with health checks, fallback logic, and cost comparison dashboard.	✅ `m6-pt03-multicloud/`

Challenges Phase (Days 24-26)

Day	Date	Task	Activity	Commit Target
24	12 Mar	**CH 1: Fault-Tolerant Agent Swarm**	🔨 Build a swarm of 10 agents where if any 3 fail, the system continues. Implement heartbeat monitoring, automatic restart of failed agents, and consensus-based result validation with Byzantine fault tolerance.	✅ `m6-ch01-fault-tolerant/`
25	15 Mar	**CH 2: Azure CI/CD Pipeline**	🔨 Create a complete CI/CD pipeline in Azure DevOps or GitHub Actions: lint → test → build Docker image → push to ACR → deploy to Container Apps → run smoke tests → rollback on failure. Include deployment gates.	✅ `m6-ch02-azure-cicd/`
26	16 Mar	**CH 3: Cross-Region Agent with Latency Optimization**	🔨 Deploy the same agent in 3 Azure regions. Build a geo-routing layer that directs users to the nearest healthy deployment. Measure and optimize end-to-end latency with Azure Traffic Manager or custom routing.	✅ `m6-ch03-cross-region/`

Deliverable Phase (Day 27)

Day	Date	Task	Activity	Commit Target
27	17 Mar	**Deliverable**	🎯 `azure-agent-platform` — Polish GitHub repo + Azure deployment with IBM multi-agent patterns, Prompt Flow definitions, SK integration, Dockerfiles, Bicep/Terraform templates, GitHub Actions CI/CD, Azure Monitor dashboards, and comprehensive deployment guide with cost analysis.	✅ `m6-deliverable/`

M7 — Orchestration + Docker + LangSmith

Duration: 27 days | **18 Mar → 23 Apr 2027**

> Resources: Coursera AI Agents (5 modules) + Docker YouTube + LangSmith Docs

Weekly Objectives

Week	Dates	Days	Objective for the week
W1	18 Mar – 24 Mar	5	Coursera — GenAI Basics Review; Coursera — Agentic Workflows; Coursera — Single Agent Systems; Coursera — Multi-Agent Orchestration; Cour...
W2	25 Mar – 31 Mar	5	Coursera — Capstone (Part 1); Coursera — Capstone (Part 2); Coursera — Capstone (Part 3); Docker — Basics & Containers; Docker — Multi-st...
W3	01 Apr – 07 Apr	5	Docker — docker-compose; Docker — Agent Containerization; LangSmith — Tracing & Debugging; LangSmith — Datasets & Evaluations; LangSmith ...
W4	08 Apr – 14 Apr	5	LangSmith — Monitoring & Alerts; LangSmith — A/B Testing & Review; Orchestrated Multi-Agent System; Dockerized Agent Stack; LangSmith Eva...
W5	15 Apr – 21 Apr	5	Complex Orchestration with Retry Logic; Production Docker Setup; Full Observability Stack; GitOps Deployment Pipeline; Auto-Remediation A...
W6	22 Apr – 23 Apr	2	Cost-Optimized Model Router; Deliverable (milestone capstone)

Learning Phase (Days 1-17)

Day	Date	Topic	Activity	Commit Target
1	18 Mar	Coursera — GenAI Basics Review	📺 Module 1: LLM fundamentals, prompting, token economics	✅ `m7-d01-coursera-m1.md` — LLM fundamentals review
2	19 Mar	Coursera — Agentic Workflows	📺 Module 2: Agent design, ReAct, tool use, planning vs acting	✅ `m7-d02-coursera-m2.py` — workflow implementations
3	22 Mar	Coursera — Single Agent Systems	📺 Module 3: Building robust single agents, error handling, retries	✅ `m7-d03-coursera-m3.py` — robust agent patterns
4	23 Mar	Coursera — Multi-Agent Orchestration	📺 Module 4: Orchestration patterns, inter-agent communication, consensus	✅ `m7-d04-coursera-m4.py` — orchestration patterns
5	24 Mar	Coursera — Advanced Topics	📺 Module 5: Evaluation, safety, deployment, production considerations	✅ `m7-d05-coursera-m5.md` — production checklist
6	25 Mar	Coursera — Capstone (Part 1)	💻 Start final project: design multi-agent orchestration system	✅ `m7-d06-capstone-design.md` — capstone architecture
7	26 Mar	Coursera — Capstone (Part 2)	💻 Implement core orchestrator and agent definitions	✅ `m7-d07-capstone-core/` — core implementation
8	29 Mar	Coursera — Capstone (Part 3)	💻 Add evaluation, monitoring, documentation,	✅ `m7-d08-capstone-final/` — complete

Day	Date	Topic	Activity	Commit Target
			and submit	capstone
9	30 Mar	Docker — Basics & Containers	📺 Containers, images, Dockerfile basics, `docker build`, `docker run`	✅ `m7-d09-docker-basics/` — first containerized app
10	31 Mar	Docker — Multi-stage Builds	📺 Optimization, layer caching, small images, distroless bases	✅ `m7-d10-docker-multistage/` — optimized Dockerfile
11	01 Apr	Docker — docker-compose	📺 Multi-container apps, networks, volumes, environment variables	✅ `m7-d11-docker-compose/` — multi-service compose file
12	02 Apr	Docker — Agent Containerization	💻 Dockerize your LangGraph + FastAPI agent with proper configs	✅ `m7-d12-agent-docker/` — containerized agent stack
13	05 Apr	LangSmith — Tracing & Debugging	📺 Trace LLM calls, tool calls, agent loops, nested tracing	✅ `m7-d13-ls-tracing.py` — traced agent implementation
14	06 Apr	LangSmith — Datasets & Evaluations	📺 Create datasets, run batch evaluations, custom metrics	✅ `m7-d14-ls-evals.py` — evaluation dataset + run
15	07 Apr	LangSmith — Hub & Prompt Management	📺 Prompt versioning, collaboration, A/B testing prompts	✅ `m7-d15-ls-hub.py` — prompt version management

Day	Date	Topic	Activity	Commit Target
16	08 Apr	LangSmith — Monitoring & Alerts	📺 Production monitoring, feedback collection, regression alerts	✅ `m7-d16-ls-monitor.py` — monitoring dashboard setup
17	09 Apr	LangSmith — A/B Testing & Review	📺 Experimentation, comparison metrics, statistical significance + 📝 Review all	✅ `m7-d17-ls-abtest.py` — A/B test implementation

Worked Examples Phase (Days 18-20)

Day	Date	Task	Activity	Commit Target
18	12 Apr	**WE 1: Orchestrated Multi-Agent System**	🔨 Build a complex orchestration with 6 agents using a central orchestrator. Implement: task decomposition, parallel execution, result aggregation, and error recovery. Use LangGraph for workflow + custom orchestrator logic with health checks.	✅ `m7-we01-orchestrator/`
19	13 Apr	**WE 2: Dockerized Agent Stack**	🔨 Create a complete `docker-compose.yml` with: FastAPI agent service, Redis cache, Pinecone/Postgres vector DB, LangSmith tracing, and Nginx reverse proxy. Include health checks, graceful shutdown, and log aggregation.	✅ `m7-we02-docker-stack/`
20	14 Apr	**WE 3: LangSmith Evaluation Pipeline**	🔨 Build an automated evaluation pipeline: define 100 test cases → run through agent → collect traces → compute metrics (latency, token usage, accuracy) → generate HTML dashboard → alert on regression via email/Slack.	✅ `m7-we03-ls-pipeline/`

Practice Tasks Phase (Days 21-23)

Day	Date	Task	Activity	Commit Target
21	15 Apr	**PT 1: Complex Orchestration with Retry Logic**	🔨 Build an orchestrator that handles agent failures with exponential backoff, circuit breaker pattern, and fallback to simpler agents. Log all decisions and retry attempts with structured logging.	✅ `m7-pt01-complex-orch/`
22	16 Apr	**PT 2: Production Docker Setup**	🔨 Create production-ready Docker images for your agent: multi-stage build, non-root user, health checks, structured JSON logging, and secrets management via Docker secrets or environment variables with validation.	✅ `m7-pt02-prod-docker/`
23	19 Apr	**PT 3: Full Observability Stack**	🔨 Integrate LangSmith + Prometheus + Grafana. Collect metrics: request rate, error rate, latency, token cost per request. Create Grafana dashboards and set up PagerDuty/Slack alerts for SLO breaches.	✅ `m7-pt03-observability/`

Challenges Phase (Days 24-26)

Day	Date	Task	Activity	Commit Target
24	20 Apr	**CH 1: GitOps Deployment Pipeline**	🔨 Implement GitOps with ArgoCD or Flux: push to Git → automated test → build image → update K8s manifests → deploy to staging → run smoke tests → promote to production with approval gates.	✅ `m7-ch01-gitops/`
25	21 Apr	**CH 2: Auto-Remediation Agent**	🔨 Build a self-healing agent that monitors its own error rate via LangSmith. When errors spike, it automatically adjusts temperature, switches models, or falls back to simpler prompts. Log all remediation actions and their outcomes.	✅ `m7-ch02-auto-remediate/`
26	22 Apr	**CH 3: Cost-Optimized Model Router**	🔨 Create an intelligent router that selects the cheapest model capable of handling each request. Maintain a performance registry (accuracy vs cost) and retrain the router weekly based on LangSmith evaluation data.	✅ `m7-ch03-cost-router/`

Deliverable Phase (Day 27)

Day	Date	Task	Activity	Commit Target
27	23 Apr	**Deliverable**	🎯 `production-agent-platform` — Polish GitHub repo with production multi-agent orchestration, complete docker-compose + individual Dockerfiles, LangSmith evaluation datasets + test suites, Prometheus/Grafana configs + dashboards, K8s manifests, and comprehensive production runbook with troubleshooting guide.	✅ `m7-deliverable/`

M8 — Capstone Projects

Duration: 36 days | **26 Apr → 11 Jun 2027**

> **Goal:** Build 3 portfolio projects that demonstrate end-to-end AI Engineering skills. Each project gets dedicated design, build, deploy, and polish days.

📅 Weekly Objectives

Week	Dates	Days	Objective for the week
W1	26 Apr – 30 Apr	5	Design; Core; Tools; Backend; Memory
W2	03 May – 07 May	5	Frontend; Docker; Deploy; Testing; Polish
W3	10 May – 14 May	5	Design; GitHub; Analyzer; Reviewer; Orchestrator
W4	17 May – 21 May	5	Backend; Deploy; Polish; Design; Ingestion
W5	24 May – 28 May	5	RAG; Backend; Auth; Frontend; Docker
W6	31 May – 04 Jun	5	LangSmith; Azure; Load Test; Portfolio; Blog 1
W7	04 Jun – 10 Jun	5	Blog 2; Blog 3; Profile; Demos; Mock
W8	11 Jun	1	Final

Project 1: AI Research Agent (Days 1-10)

Day	Date	Phase	Activity	Commit Target
1	26 Apr	Design	📝 Define architecture, tech stack, data flow, API endpoints, UI mockups	✅ `m8-p1-design.md` — complete project spec
2	27 Apr	Core	💻 Build web search + summarization agent with LangGraph	✅ `m8-p1-core/` — agent core implementation
3	28 Apr	Tools	💻 Add multi-source search (DuckDuckGo, arXiv, news), fact-checking	✅ `m8-p1-tools/` — tool integrations
4	29 Apr	Backend	💻 FastAPI REST API with async endpoints, Pydantic models, error handling	✅ `m8-p1-api/` — complete backend
5	30 Apr	Memory	💻 Add vector memory with Pinecone, conversation history, context management	✅ `m8-p1-memory/` — memory system
6	03 May	Frontend	💻 Simple React/Gradio UI for interaction, streaming responses	✅ `m8-p1-ui/` — user interface
7	04 May	Docker	💻 Containerize with Docker, docker-compose for full stack	✅ `m8-p1-docker/` — containerization
8	05 May	Deploy	💻 Deploy to Azure Container Apps or similar cloud platform	✅ `m8-p1-deploy/` — live deployment
9	06 May	Testing	💻 Unit tests, integration tests, load tests with k6/locust	✅ `m8-p1-tests/` — test suite

Day	Date	Phase	Activity	Commit Target
10	07 May	Polish	📝 Final README, architecture diagram, demo script, code refactor	✅ `m8-p1-final/` — polished project

Project 2: Code Review Multi-Agent (Days 11-18)

Day	Date	Phase	Activity	Commit Target
11	10 May	Design	📝 Multi-agent architecture for code review, GitHub API integration plan	✅ `m8-p2-design.md` — architecture document
12	11 May	GitHub	💻 GitHub API integration, webhook handling, PR diff fetching	✅ `m8-p2-github/` — GitHub integration
13	12 May	Analyzer	💻 Static analysis agent, bug detection, code smell identification	✅ `m8-p2-analyzer/` — analysis agent
14	13 May	Reviewer	💻 Code review agent with inline suggestions, style checks, best practices	✅ `m8-p2-reviewer/` — review agent
15	14 May	Orchestrator	💻 Multi-agent coordination with CrewAI, report generation, severity scoring	✅ `m8-p2-orch/` — orchestration layer
16	17 May	Backend	💻 FastAPI backend, GitHub App integration, OAuth, webhooks	✅ `m8-p2-api/` — complete backend
17	18 May	Deploy	💻 Docker, deploy to cloud, configure GitHub App settings	✅ `m8-p2-deploy/` — live deployment
18	19 May	Polish	📝 Tests, README, demo video script, error handling improvements	✅ `m8-p2-final/` — polished project

Project 3: RAG-Powered API (Days 19-28)

Day	Date	Phase	Activity	Commit Target
19	20 May	Design	📝 FastAPI + LangChain + Pinecone architecture, auth, multi-tenancy plan	✅ `m8-p3-design.md` — technical specification
20	21 May	Ingestion	💻 PDF/text/DOCX ingestion, async processing queue, chunking pipeline	✅ `m8-p3-ingest/` — document ingestion
21	24 May	RAG	💻 Retrieval, generation, citation, streaming response implementation	✅ `m8-p3-rag/` — RAG pipeline
22	25 May	Backend	💻 Document upload, query, admin endpoints with validation	✅ `m8-p3-api/` — REST API
23	26 May	Auth	💻 API keys, JWT tokens, user isolation, rate limits per tenant	✅ `m8-p3-auth/` — authentication system
24	27 May	Frontend	💻 Document management UI, chat interface with streaming, admin dashboard	✅ `m8-p3-ui/` — web interface
25	28 May	Docker	💻 Full stack containerization with docker-compose, Nginx, Redis	✅ `m8-p3-docker/` — container stack
26	31 May	LangSmith	💻 Tracing integration, evaluation suite, monitoring setup	✅ `m8-p3-langsmith/` — observability
27	01 Jun	Azure	💻 Deploy to Azure Container Apps, configure CI/CD, custom domain	✅ `m8-p3-azure/` — cloud deployment
28	02 Jun	Load Test	💻 k6/locust load tests, optimization, caching layer, connection pooling	✅ `m8-p3-loadtest/` — performance testing

Portfolio Polish & Launch (Days 29-36)

Day	Date	Phase	Activity	Commit Target
29	03 Jun	Portfolio	📝 Unified README across all 3 repos, architecture diagrams, tech stack summary	✅ `m8-portfolio.md` — portfolio overview
30	04 Jun	Blog 1	📝 Write technical blog on Project 1 (Research Agent) — architecture, challenges, learnings	✅ `m8-blog1.md` — published blog post
31	04 Jun	Blog 2	📝 Write technical blog on Project 2 (Code Review) — multi-agent patterns, GitHub integration	✅ `m8-blog2.md` — published blog post
32	07 Jun	Blog 3	📝 Write technical blog on Project 3 (RAG API) — production RAG, Azure deployment, scaling	✅ `m8-blog3.md` — published blog post
33	08 Jun	Profile	📝 Update resume with AI Engineer focus, LinkedIn, GitHub profile, pin all 3 projects	✅ `m8-profile/` — updated profiles
34	09 Jun	Demos	📼 Record 3 demo videos (5 min each), upload to YouTube/Loom, embed in READMEs	✅ `m8-demos/` — demo videos
35	10 Jun	Mock	📝 Practice explaining architecture decisions, system design, trade-offs	✅ `m8-mock-int.md` — interview prep notes
36	11 Jun	Final	💻 Final code review, ensure all repos green, all tests passing, all docs complete	✅ `m8-final/` — final polish

Capstone Deliverables

🎯 3 Production-Grade Portfolio Projects:

Project 1: `ai-research-agent`

- **Stack:** LangGraph, FastAPI, Pinecone, DuckDuckGo Search, OpenAI
- **Features:** Multi-source research, citation tracking, vector memory, REST API, streaming UI
- **Deploy:** Docker + Azure Container Apps
- **Repo:** Clean code, tests >80%, README, demo video, live URL

Project 2: `code-review-multi-agent`

- **Stack:** CrewAI, FastAPI, GitHub API, LangSmith
- **Features:** PR analysis, multi-agent review, inline suggestions, GitHub App integration, severity scoring
- **Deploy:** Docker + Azure
- **Repo:** Clean code, tests, README, demo video, live URL

Project 3: `rag-api-platform`

- **Stack:** FastAPI, LangChain, Pinecone, Docker, Azure, LangSmith, Redis
- **Features:** Document upload (PDF/DOCX), chunking, embedding, RAG query with citation, auth, multi-tenancy, admin dashboard
- **Deploy:** Docker Compose / Azure Container Apps
- **Repo:** Clean code, tests, API docs, README, demo video, live URL

🎯 Additional Deliverables:

- 3 technical blog posts (published on Medium/Dev.to/Hashnode)
- Updated resume with AI Engineer focus
- Updated LinkedIn with project showcases
- GitHub profile pinned with all 3 projects

- Demo videos (5 min each) on YouTube/Loom
- Mock interview preparation completed

Daily Commit Convention

```
m{MILESTONE}-d{DAY}-{description}.py
m{MILESTONE}-we{NUM}-{description}/
m{MILESTONE}-pt{NUM}-{description}/
m{MILESTONE}-ch{NUM}-{description}/
m{MILESTONE}-deliverable/
m{MILESTONE}-p{NUM}-{phase}/
```

Examples:

- `m0-d01-regex-basics.py`
- `m3-we02-lg-statemachine/`
- `m8-p1-core/`

Weekly Review Checklist (Every Friday)

- [ ] All daily commits pushed to GitHub
- [ ] Week's code reviewed and refactored where needed
- [ ] Notes updated in `notes/` folder
- [ ] Blockers documented with action items for next week
- [ ] Next week's plan reviewed and adjusted if needed
- [ ] 1 LinkedIn post about what you learned this week
- [ ] Green GitHub graph maintained (no blank days)

Buffer Day Policy

- **Buffer days** (Saturdays, Sundays, hackathons) are for:
  - Catching up on missed tasks

    - Refactoring previous week's code
    - Deep reading of papers or documentation
    - Rest and recovery (important!)
- **Do NOT skip ahead** to next milestone content
- **Do NOT cram** multiple tasks into one day
- If a task takes more than 1 day, extend into the next available working day

---

Tools & Accounts Setup (Complete ASAP if not already done)

- [ ] GitHub account + empty repos created for each milestone
- [ ] OpenAI API key (or Azure OpenAI access)
- [ ] Pinecone account (free tier)
- [ ] LangSmith account (free tier)
- [ ] HuggingFace account + token
- [ ] Docker Desktop installed and running
- [ ] Python 3.11+ installed with `venv` support
- [ ] VS Code + extensions (Python, Pylance, Docker, Markdown, GitLens)
- [ ] Azure account (free tier + $200 credit)
- [ ] Coursera audit enrollments (IBM, Orchestration courses)
- [ ] DeepLearning.AI free enrollments (LangGraph, CrewAI)
- [ ] Great Learning free enrollment (Agentic AI)
- [ ] Notion/Obsidian for notes and knowledge base
- [ ] Postman or Insomnia for API testing

Success Metrics

Milestone	Completion Criteria
M0	All PY4E Ch 11-16 exercises done + 3 WE + 3 PT + 3 CH completed + `agentic-python-toolkit` repo live
M1	All 21 MS lessons complete + 3 WE + 3 PT + 3 CH completed + `genai-foundation-toolkit` repo live
M2	Agentic AI + FastAPI courses done + 3 WE + 3 PT + 3 CH completed + `agentic-fastapi-starter` repo live
M3	12 MS lessons + LangGraph course done + 3 WE + 3 PT + 3 CH completed + `langgraph-agent-suite` repo live
M4	CrewAI + HF courses done + HF certification obtained + 3 WE + 3 PT + 3 CH completed + `multi-framework-agent-suite` repo + HF Space live
M5	RAG series + Pinecone done + 3 WE + 3 PT + 3 CH completed + `production-rag-system` repo live
M6	IBM course + Azure path done + 3 WE + 3 PT + 3 CH completed + `azure-agent-platform` repo + Azure deployment live
M7	Coursera course + Docker + LangSmith done + 3 WE + 3 PT + 3 CH completed + `production-agent-platform` repo live
M8	3 deployed projects + 3 blog posts + updated resume/LinkedIn + demo videos + mock interviews done

Gap Analysis — AI Engineer JD vs This Roadmap

Required Skill (Job JD)	Covered By
LangChain, LangGraph, LlamaIndex	M3, M4, M5, M6, M7
Multi-agent (CrewAI, AutoGen/AG2)	M3, M4, M6
RAG pipelines + embeddings	M1, M5, M8-P3
Vector DBs (Pinecone, Weaviate)	M5, M8-P3
FastAPI REST backend	M2, M7, M8-P1/P2/P3
Azure OpenAI + AI Foundry	M3, M6, M8
Docker basics + production	M7, M8
LangSmith observability	M3, M7, M8
Python proficiency	M0, M8
Portfolio proof	M8 (3 projects + blogs + demos)

Final Notes

- **No parallel items** — Finish one milestone, start next. No parallel overload.
- **2 hours/day** — Consistency beats intensity. Daily habit > weekend cramming.
- **Commit daily** — Green GitHub graph = proof of work. Har din kuch na kuch push hona chahiye.
- **Build in public** — Share weekly progress on LinkedIn/Twitter. Tag relevant frameworks.
- **Hackathons are learning** — Apply what you've learned, don't pause completely. Use hackathon days as applied practice.

- **One task = one day minimum** — Koi jaldi nahi. Quality > speed. Agar task zyada waqt ley to next day continue kero.
- **Buffer days are sacred** — Use them to catch up, refactor, or rest. Don't skip ahead.
- **Document everything** — Har milestone ka apna README ho. Future mein yeh portfolio ka hisa banay ga.
- **Ask for help** — Stack Overflow, Reddit r/LangChain, Discord servers, GitHub issues. Community se seekho.

Bismillah. Restarted on 12th August from Networked Programs. No excuses. Consistency is the key.

> *"The only way to do great work is to love what you do."* — Steve Jobs

---

Roadmap generated on 1 Aug 2026. Last updated: 12 Aug 2026 (re-scheduled from Networked Programs / Day 3). Total working days: 219 | Calendar span: 10 Aug 2026 → 11 Jun 2027